Practical 1

Ethan Van Eyden (u25260244)

Lesego Tebeile (u25143230)

Task 1

1.5

The open-closed principle refers to open to extension, closed to modification. A single if-else chain violates this principle since to extend a program you would have to modify an existing part of your program. Using the factory method avoids this because you just must create a new concrete subclass of your abstract factory class. Existing classes do not need to be modified at all so the system can be extended by adding classes instead of modifying existing classes. This reduces the risk of introducing bugs to working code and makes code maintainability easier.

1.6

- Connector – Product: Defines the interface or abstract class for objects the factory creates
- CsvConnector – Concrete Product: Implements the Connector interface
- Connector Factory – Creator: Declares the factory method that returns a Connector
- CsvFactory – Concrete Creator: Overrides the factory method to create and return a CsvConnector

Task 2

2.5

A shallow copy performs member-wise copying of an object. For pointer members, only the memory address is copied, so both objects point to the same dynamically allocated memory. A deep copy allocates new memory and copies the contents into it, so each

object owns its own separate copy of the data. clone() must produce a deep copy because it should create an independent object. If the clone shared memory with the prototype, changes to one object could affect the other, and deleting one object could lead to dangling pointers or double deletion.

2.6

The registry returns clone() objects instead of the stored prototype pointer because the registry should provide independent transformation objects while hiding the concrete creation process. The registry only works with the abstract Transformation interface and allows the prototype object to create copies of itself. If multiple pipelines shared the same prototype object, changes made by one pipeline could affect the others. It would also create ownership problems, potentially causing double deletion or dangling pointers when multiple owners try to manage the same object.

Task 3

3.5

run() is deliberately not virtual because the Template Method pattern requires the base class to control the fixed algorithm skeleton. All pipelines must execute the lifecycle in the same order: connect, extract, transform and load. Subclasses should only override specific primitive operations such as extract() and load(). If run() could be overridden, a subclass could change the order of operations or remove required steps, breaking the pipeline lifecycle and the intended design.

3.6

- run() is the Template Method because it defines the fixed pipeline lifecycle: connect, extract, transform and load.

- connect() and transform() are concrete steps because they are implemented in the base class.
- extract() and load() are the primitive operations because subclasses provide their own implementations.

The Factory Method and the Template Method solve different problems. The Factory Method controls object creation by allowing different factories to create different Connector objects without changing the Pipeline. The Template Method controls the execution algorithm by fixing the pipeline lifecycle while allowing subclasses to customise specific steps. They do not overlap because one manages object creation and the other manages algorithm structure.

Task 4

4.4

The wide interface gives full access to the stored state. Only the originator may use it because it allows the originator to save its state into the memento and restore its state from the memento. The narrow interface exposes little or no information about the mementos contents. The caretaker uses it because it only needs to store, retrieve and pass mementos back to the originator without the need to inspect its contents.

4.5

1. The pipeline calls createCheckpoint() immediately before entering the load() stage, saving stage=3 and the transformed records
2. The RunCheckpoint is then stored in the history vector when it's passed into the CheckpointManager's save() method.
3. If something goes wrong then then the undo() method is called returning the most recent checkpoint
4. Restore() is then called with the RunCheckpoint pointer returned passed in which sets the stage and records equal to that of the checkpoint

5. The pipeline then re-invokes load() to retry only the loading step, avoiding re-extraction and re-transformation

4.6

Pipeline – The originator

RunCheckpoint – Memento

CheckpointManager - Caretaker